NEURON

Documentação Completa da Arquitetura do MVP (versão atualizada)

React + Spring Boot + PostgreSQL/pgvector + FastAPI + LangChain/LangGraph

Status: arquitetura-base aprovada. Este documento consolida as decisões técnicas tomadas até esta etapa e identifica explicitamente o que ainda está em aberto.

> **O que mudou nesta versão:** (1) o monitor humano está fora do escopo atual; (2) entra a arquitetura da prática com questões, apenas em disciplinas com prova (seção 7); (3) entra o feedback da plataforma (seção 12). Itens marcados como **(proposta)** ainda precisam ser validados pela equipe; os pontos em aberto estão na seção 30.

1. Objetivo arquitetural

A arquitetura do NEURON foi desenhada para separar a aplicação web, as regras de negócio e o serviço especializado de IA. O Spring Boot é o backend principal e funciona como porta de entrada para o front-end. O serviço Python é interno e dedicado ao Monitor Virtual e à prática com questões.

Princípio central

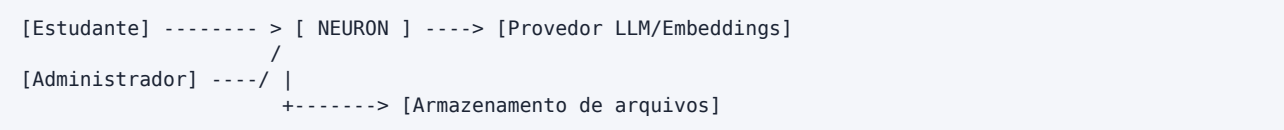
```
React
 |
 v
Spring Boot
 |----> PostgreSQL + pgvector
 |
 +----> FastAPI
          |
          +--> LangChain / LangGraph
                  |
                  +--> LLM / Embeddings
```

O React não acessa diretamente o FastAPI nem recebe chaves de provedores de IA. Isso reduz acoplamento, concentra autenticação e autorização no backend principal e permite substituir componentes internos sem alterar a interface pública.

2. Stack arquitetural aprovada

Camada	Tecnologia	Responsabilidade
Front-end	React	Interface web responsiva, navegação, formulários, chat, prática com questões, feedback e exibição de fontes.
Backend principal	Java + Spring Boot	API pública, regras de negócio, segurança, CRUD e orquestração das integrações.
Banco	PostgreSQL + pgvector	Dados relacionais e índice vetorial derivado para o RAG.
Serviço de IA	Python + FastAPI	API interna do Monitor Virtual e da prática com questões.
RAG/agente	LangChain + LangGraph	Ingestão, recuperação, avaliação de contexto, reescrita e geração.
LLM/Embeddings	Em aberto	Modelo de linguagem e geração de embeddings.
Arquivos	Em aberto	Armazenamento dos materiais; Google Drive é uma possibilidade ainda não fechada.
Deploy	Em aberto	Hospedagem final ainda será comparada e aprovada.

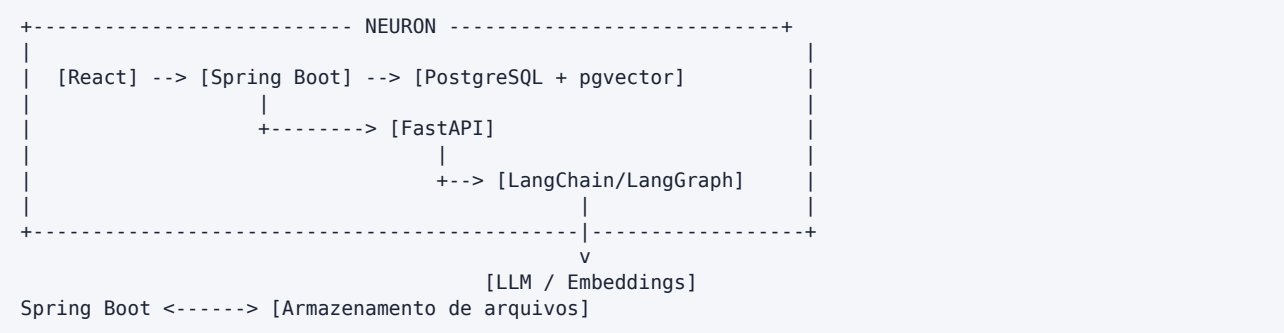
3. Visão de contexto



O Estudante consulta conteúdos, utiliza o Monitor Virtual, pratica com questões (nas disciplinas com prova) e envia feedback sobre a plataforma. O Administrador publica e organiza conteúdo, gerencia a base de conhecimento e consulta o feedback. Serviços externos de IA e armazenamento são dependências controladas pelo backend/serviço interno.

Fora do escopo atual: o monitor humano (monitoria entre estudantes, candidatura, agendamentos, mensagens e avaliações de monitores) fica para uma fase futura e não tem componentes nesta arquitetura.

4. Visão de contêineres



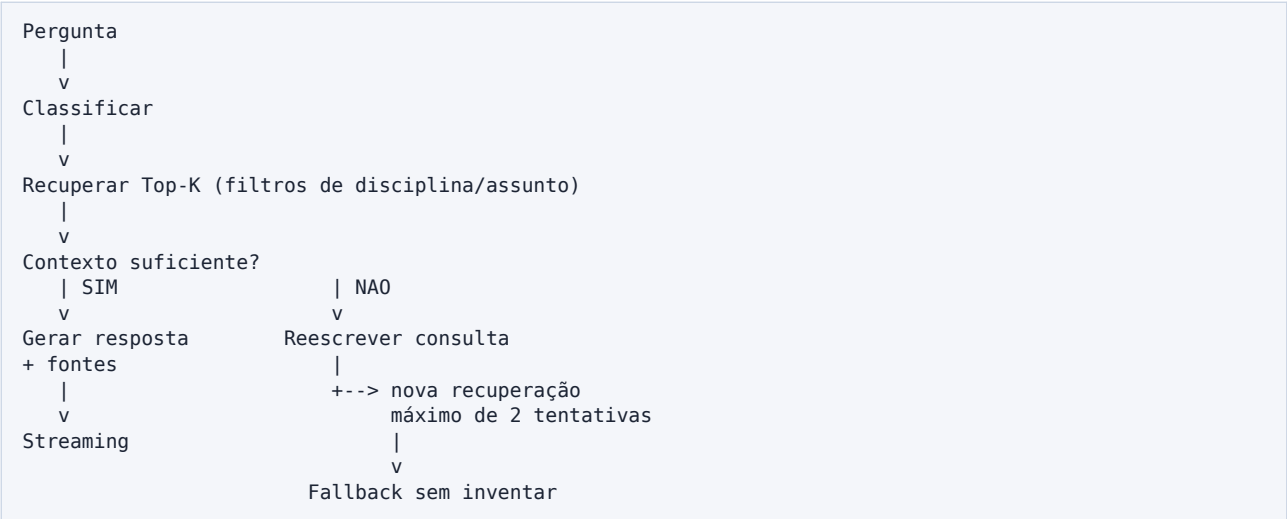
Embora PostgreSQL e pgvector apareçam conceitualmente como responsabilidades distintas, a decisão do MVP é utilizar o pgvector como extensão do próprio PostgreSQL. Assim, a equipe administra um único SGBD.

5. Limites e responsabilidades

Componente	Deve fazer	Não deve fazer
React	Consumir /api/v1, controlar estado de UI, renderizar streaming, fontes, prática e feedback.	Acessar FastAPI, banco ou chaves do LLM diretamente.
Spring Boot	Autenticar, autorizar, validar, executar regras (incluindo a disponibilidade da prática), CRUD, registrar feedback e chamar o serviço de IA.	Implementar o pipeline LangChain/LangGraph.
FastAPI	Executar operações internas de chat, prática e ingestão/RAG.	Ser exposto diretamente ao navegador.
PostgreSQL	Persistir dados relacionais do sistema.	Ser substituído pelo índice vetorial como fonte de verdade.
pgvector	Armazenar embeddings/chunks e executar similaridade.	Ser tratado como banco mestre dos materiais.
LLM	Gerar resposta a partir do contexto fornecido.	Receber credenciais ou chamadas diretamente do front-end.

6. Arquitetura do Monitor Virtual

O Monitor Virtual usa RAG. A disciplina é obrigatória para o chat; o assunto é opcional. A recuperação deve respeitar esses filtros e utilizar apenas materiais autorizados como fonte do Monitor Virtual.

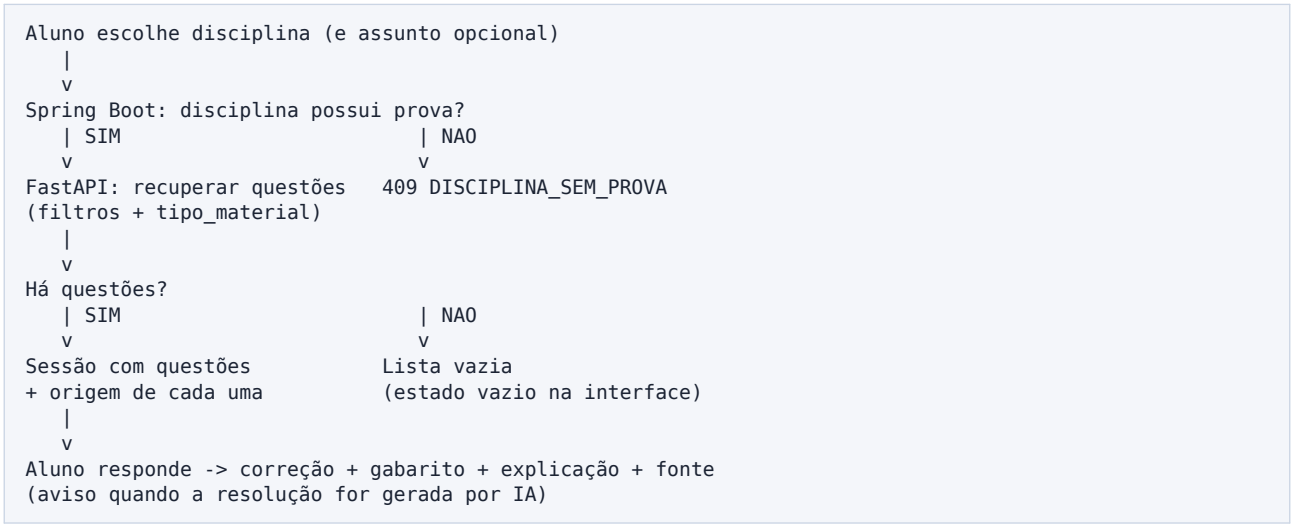


Fallback previsto: quando não houver contexto suficiente, o sistema informa que não encontrou informações suficientes nos materiais da disciplina, em vez de produzir uma resposta apresentada como fundamentada.

7. Arquitetura da prática com questões (novo)

A prática permite que o aluno resolva questões no padrão de cobrança dos professores, a partir das listas de exercícios e provas cadastradas. Ela reaproveita a base vetorial e o serviço de IA, com regras próprias.

- **Disponibilidade:** somente em disciplinas com prova (campo possui_prova, proposta). O Spring Boot valida isso antes de chamar a IA; para disciplinas avaliadas por projetos, devolve erro 409 com código DISCIPLINA_SEM_PROVA, e a interface informa o motivo.
- **Fontes das questões:** apenas materiais dos tipos LISTA_EXERCICIOS e PROVA_ANTIGA marcados como disponíveis para prática e com autorização do professor.
- **Recuperação:** filtrada por disciplina (obrigatória), assunto (opcional) e tipo_material.
- **Origem visível:** cada questão devolve a origem (título do material, página e número da questão), usada no selo exibido ao aluno.
- **Gabarito:** usado como veio do material quando existir. Quando não existir, a resolução gerada pela IA deve ser sinalizada como tal, com aviso para conferir com o professor.
- **Estado vazio:** sem exercícios cadastrados para o assunto, a resposta é uma lista vazia e a interface informa isso, sem inventar questões.
- **Sem histórico:** a sessão de prática é efêmera, sem persistência no banco relacional (mesmo princípio do thread_id do chat).
- **Em aberto:** se a primeira entrega apenas exibe questões existentes ou também gera questões novas no mesmo padrão (seção 30).



8. Fluxo de chat ponta a ponta

```
Aluno
 |
 | pergunta
 v
React
 |
 | POST /api/v1/monitor/chat
 v
Spring Boot
 | autentica / valida / rate limit
 |
 | POST /internal/v1/chat
 v
FastAPI / LangGraph
 |
 | busca vetorial filtrada
 v
PostgreSQL + pgvector
 |
 | Top-K chunks
 v
FastAPI -> LLM
 |
 | tokens + fontes
 v
Spring Boot
 |
 | SSE
 v
React -> resposta progressiva ao aluno
```

Meta inicial: iniciar a resposta do Monitor Virtual em até aproximadamente 5 segundos, valor que ainda deverá ser validado por testes reais de infraestrutura, provedor e volume de documentos.

9. Streaming SSE

A API pública do Monitor Virtual retorna text/event-stream. Como a requisição é POST com JSON, o front-end deverá consumir o fluxo com fetch/ReadableStream ou biblioteca compatível com SSE via POST; EventSource nativo não deve ser assumido para esse endpoint.

```
event: token
data: {"texto":"A arquitetura"}

event: token
data: {"texto":" de Von Neumann..."}

event: sources
data: {"fontes":[...]}

event: done
data: {}
```

10. Ingestão e indexação de materiais



Ao cadastrar ou alterar um Material marcado como fonte do Monitor Virtual, o backend solicita indexação. Ao desativá-lo, os chunks derivados devem ser removidos. Ao substituir o documento, os chunks antigos são removidos e o material é reindexado.

Listas de exercícios e provas (novo): seguem o mesmo fluxo. Para esses materiais, o chunking deve preservar a estrutura de questões (uma questão por chunk, com número da questão e gabarito quando houver); esse detalhe será validado na etapa de materiais. Materiais disponíveis para prática também são indexados, mesmo que não sejam fonte do Monitor Virtual (proposta).

11. Metadados vetoriais

Campo	Uso
material_id	Vincular o chunk ao Material mestre.
periodo_id	Filtro e rastreabilidade.
disciplina_id	Filtro obrigatório de recuperação conforme contexto do aluno.
assunto_id	Filtro opcional por assunto.
tipo_material	Identificar slide, lista, prova, resumo etc.; também separa aulas de questões na prática.
titulo_material	Montagem das fontes apresentadas ao aluno.
nome_arquivo	Identificação do documento original, quando aplicável.
pagina	Referência da página de origem.
chunk_index	Posição do chunk no documento.
numero_questao (proposta)	Número da questão em listas e provas, usado na origem exibida.
tem_gabarito (proposta)	Indica se o gabarito veio do material do professor.

O banco relacional é o dono dos dados. O conteúdo vetorial pode ser reconstruído a partir dos materiais cadastrados. A disponibilidade para prática é verificada no relacional (material e disciplina).

12. Feedback da plataforma (novo)

O estudante autenticado envia uma avaliação e um comentário sobre a plataforma. O Spring Boot valida e grava o registro na tabela feedback_plataforma; o administrador consulta os retornos por endpoint administrativo. Esse fluxo não passa pelo serviço de IA. Ele substitui a tela de avaliações de monitores do protótipo anterior, que pertencia ao monitor humano.

O formato exato (escala da nota, categorias, comentário livre e anonimato) está em aberto (seção 30).

13. Contrato público - React <-> Spring Boot

Prefixo aprovado: /api/v1.

Método	Endpoint	Finalidade
POST	/auth/login	Autenticar usuário.
POST	/auth/logout	Encerrar autenticação conforme estratégia escolhida.
GET	/auth/me	Consultar usuário autenticado.
GET	/periodos	Listar períodos ativos.
GET	/periodos/{id}	Consultar período.

Método	Endpoint	Finalidade
GET	/periodos/{periodoId}/disciplinas	Listar disciplinas do período.
GET	/disciplinas/{id}	Detalhar disciplina (inclui o indicador possuiProva, proposta).
GET	/disciplinas/{disciplinaId}/assuntos	Listar assuntos.
GET	/disciplinas/{disciplinaId}/materiais	Listar materiais; assuntoId opcional.
GET	/pesquisa?q=...	Pesquisar Disciplina, Assunto e Material.
POST	/monitor/chat	Chat do Monitor Virtual com streaming SSE.
POST	/pratica/sessoes *(proposta)*	Iniciar prática: disciplinaId, assuntoId opcional, origem e quantidade; retorna as questões com origem. Erro 409 se a disciplina não tiver prova.
POST	/pratica/sessoes/{sessaoId}/respostas *(proposta)*	Enviar a resposta de uma questão; retorna correção, gabarito, explicação, fonte e indicação de resolução gerada por IA.
POST	/feedback *(novo)*	Enviar feedback da plataforma (usuário autenticado).

O sessaoId é um identificador efêmero, sem tabela no banco relacional. Os formatos de requisição e resposta da prática serão detalhados na especificação OpenAPI.

14. Endpoints administrativos

```
POST   /api/v1/admin/periodos
PUT    /api/v1/admin/periodos/{id}
DELETE /api/v1/admin/periodos/{id}
POST   /api/v1/admin/disciplinas
PUT    /api/v1/admin/disciplinas/{id}
DELETE /api/v1/admin/disciplinas/{id}
POST   /api/v1/admin/assuntos
PUT    /api/v1/admin/assuntos/{id}
DELETE /api/v1/admin/assuntos/{id}
POST   /api/v1/admin/materiais
PUT    /api/v1/admin/materiais/{id}
DELETE /api/v1/admin/materiais/{id}
GET    /api/v1/admin/feedback            (novo)
```

Esses endpoints exigem perfil ADMINISTRADOR. O DELETE representa normalmente exclusão lógica no domínio acadêmico.

Novos campos (propostas): o cadastro de disciplina aceita possuiProva; o cadastro de material aceita disponivelParaPratica, contemGabarito e autorizacaoProfessor, além de fonteMonitorVirtual.

15. Contrato interno - Spring Boot <-> FastAPI

Prefixo aprovado: /internal/v1. Esses endpoints não são destinados ao navegador.

Método	Endpoint	Finalidade
POST	/chat	Executar o fluxo RAG e devolver streaming/resposta com fontes.

Método	Endpoint	Finalidade
POST	/materials/index	Indexar novo material.
DELETE	/materials/{materialId}	Remover chunks derivados de um material.
POST	/materials/{materialId}/reindex	Remover índice anterior e reprocessar o documento.
POST	/practice/questions *(proposta)*	Montar as questões da sessão de prática com origem.
POST	/practice/answers *(proposta)*	Corrigir a resposta e devolver gabarito, explicação e fonte.

Exemplo de requisição interna de chat

```
{
 "thread_id": "uuid-da-conversa",
 "message": "O que e arquitetura de Von Neumann?",
 "disciplina_id": 8,
 "assunto_id": 31
}
```

Exemplo de requisição interna de prática (proposta)

```
{
 "sessao_id": "uuid-da-sessao",
 "disciplina_id": 3,
 "assunto_id": 12,
 "origem": "MISTO",
 "quantidade": 10
}
```

A requisição de indexação deve informar também o tipo do material e se ele está disponível para prática (proposta).

16. Padrão de erros

```
{
 "status": 404,
 "codigo": "MATERIAL_NAO_ENCONTRADO",
 "mensagem": "Material nao encontrado.",
 "timestamp": "2026-10-02T20:00:00Z"
}
```

HTTP	Uso
200	Consulta/processamento bem-sucedido.
201	Recurso criado.
204	Operação concluída sem corpo.
400	Dados de entrada inválidos.
401	Usuário não autenticado.
403	Usuário autenticado sem permissão.
404	Recurso não encontrado.
409	Conflito de estado ou unicidade. Também usado quando a prática é solicitada em disciplina sem prova (código DISCIPLINA_SEM_PROVA, proposta).

HTTP	Uso
429	Rate limit do Monitor Virtual (e da prática) excedido.
500	Erro interno.
503	Serviço de IA indisponível.

Observação: a ausência de questões para um assunto não é erro; a resposta é uma lista vazia (estado vazio).

17. Organização dos repositórios/projetos

```
neuron/
├── frontend/      # React
├── backend/       # Spring Boot
└── ai-service/    # FastAPI + LangChain + LangGraph
```

A separação permite ciclos independentes de build/deploy e reduz acoplamento entre a aplicação de negócio e a experimentação com IA.

18. Estrutura recomendada do Spring Boot

```
backend/
├── pom.xml
└── src/main/
    ├── java/br/com/neuron/
    │   ├── NeuronApplication.java
    │   ├── config/
    │   ├── security/
    │   ├── exception/
    │   ├── auth/
    │   ├── usuario/
    │   ├── periodo/
    │   ├── disciplina/
    │   ├── assunto/
    │   ├── material/
    │   ├── pesquisa/
    │   ├── monitorvirtual/
    │   ├── pratica/        (novo)
    │   ├── feedback/       (novo)
    │   └── integration/
    │       └── ai/
    └── resources/
        ├── application.yml
        └── db/migration/
```

Organização por feature/domínio é preferida. Cada domínio agrupa entidade, repository, service, controller e DTOs relacionados, evitando pacotes globais gigantes.

19. Camadas do backend

```
HTTP
 |
 v
Controller
 |
 v
Service  <- regras de negocio
 |
 v
Repository
 |
 v
PostgreSQL
```

Controller: recebe e devolve HTTP, valida formato e delega. Service: concentra regras de negócio e transações. Repository: acesso a dados via Spring Data JPA. DTOs: isolam o contrato HTTP das entidades JPA.

20. Estrutura do domínio Material

```
material/
├── Material.java
├── TipoMaterial.java
├── MaterialRepository.java
├── MaterialService.java
├── MaterialController.java
└── dto/
    ├── MaterialRequest.java
    ├── MaterialResponse.java
    └── MaterialResumoResponse.java
```

O Material possui papel especial porque conecta o domínio acadêmico ao pipeline de IA. O campo fonteMonitorVirtual determina se seu conteúdo deve integrar a base de conhecimento do Monitor Virtual. **Novo (proposta):** os campos disponivelParaPratica, contemGabarito e autorizacaoProfessor controlam o uso do material na prática.

21. Integração Spring Boot -> IA

```
integration/
└── ai/
    ├── AiClient.java
    ├── AiClientConfig.java
    └── dto/
        ├── AiChatRequest.java
        ├── AiIndexRequest.java
        ├── AiPracticeRequest.java   (novo)
        └── AiSourceResponse.java
```

A comunicação HTTP com FastAPI fica encapsulada em AiClient. Services do domínio não devem conhecer detalhes de transporte, URL interna ou formato específico do framework Python.

22. Segurança arquitetural

O Spring Boot centraliza autenticação e autorização. Estudantes possuem acesso de consulta, Monitor Virtual, prática e envio de feedback; administradores possuem acesso aos endpoints de administração, incluindo a consulta ao feedback. As chaves de LLM/embeddings permanecem no ambiente do serviço Python e nunca são enviadas ao front-end.

```
security/
├── SecurityConfig.java
├── AuthenticationFilter.java
├── TokenService.java
└── UserDetailsServiceImpl.java
```

A estratégia final de autenticação - por exemplo JWT e forma de armazenamento/transporte do token - ainda deve ser aprovada antes da implementação definitiva.

23. Banco e migrations

Recomendação arquitetural: utilizar Flyway para versionar o esquema do PostgreSQL, em vez de depender da criação automática de tabelas pelo Hibernate em ambientes compartilhados.

```
resources/db/migration/
├── V1__create_initial_schema.sql
├── V2__create_indexes.sql
└── V3__...sql
```

Os campos e a tabela novos (possui_prova, campos de prática no material e feedback_plataforma) devem entrar nas migrations conforme a modelagem de dados atualizada.

24. Configuração e segredos

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
ai-service:
  base-url: ${AI_SERVICE_URL}
```

Segredos não devem ser versionados no Git. Exemplos: DB_URL, DB_USER, DB_PASSWORD, AI_SERVICE_URL e, no serviço Python, chaves do provedor de LLM/embeddings.

25. Estrutura recomendada do React

```
frontend/src/
├── app/
├── routes/
├── pages/
│   ├── auth/
│   ├── disciplinas/
│   ├── monitor-virtual/
│   ├── pratica/        (novo)
│   ├── feedback/       (novo)
│   └── admin/
├── components/
├── features/
├── services/
│   ├── api/
│   └── monitor/
├── hooks/
├── types/
└── utils/
```

O React consome somente a API Spring Boot. A área de Monitor Virtual deve isolar a lógica de streaming para que a interface não fique acoplada ao restante da aplicação.

26. Estrutura recomendada do serviço Python

```
ai-service/
├── app/
│   ├── main.py
│   ├── api/
│   │   └── internal/
│   ├── agent/
│   │   ├── graph.py
│   │   ├── state.py
│   │   └── nodes/
│   ├── rag/
│   │   ├── loaders/
│   │   ├── chunking/
│   │   ├── retrieval/
│   │   └── embeddings/
│   ├── practice/       (novo, proposta)
│   ├── prompts/
│   ├── integrations/
│   ├── schemas/
│   └── config/
└── tests/
```

LangGraph deve representar explicitamente os estados do agente: classificação, recuperação, avaliação do contexto, reescrita limitada, geração com fontes e fallback.

27. Desenvolvimento paralelo da equipe

Frente	Pode desenvolver em paralelo
React	Telas e componentes com mocks baseados no contrato /api/v1 (incluindo prática e feedback).
Spring Boot	Estrutura do projeto, segurança, migrations, CRUD, feedback e Swagger/OpenAPI.
IA	FastAPI e pipeline RAG seguindo /internal/v1; prática depois do RAG básico.
Banco	PostgreSQL, pgvector, migrations e índices.
Testes	Casos de API, autorização, fallback, validação das fontes e regra de disponibilidade da prática.

A especificação OpenAPI é o contrato entre as frentes. Alterações incompatíveis devem ser discutidas antes de serem implementadas.

28. Requisitos não funcionais arquiteturais

Aspecto	Diretriz atual
Responsividade	Front-end compatível com PC, tablet e celular.
Desempenho	Meta inicial de até ~5 s para iniciar resposta do Monitor Virtual; validar em testes.
Segurança	Controle administrativo, segredos fora do front-end e endpoints internos protegidos.
Manutenibilidade	Separação por domínio e serviços independentes.
Compatibilidade	Principais navegadores modernos.
Rate limit	Limitar requisições do Monitor Virtual (e da prática) por usuário; política quantitativa ainda em aberto.
Reconstrução	Índice vetorial deve ser reconstruível a partir do banco relacional e materiais.

29. Decisões aprovadas

- Front-end em React.
- Backend principal em Java + Spring Boot.
- Serviço de IA independente em Python + FastAPI.
- LangChain + LangGraph para RAG/agente.
- PostgreSQL + pgvector no mesmo SGBD para o MVP.
- React chama apenas Spring Boot.
- Spring Boot chama o FastAPI por API interna.
- API pública versionada em /api/v1.
- API interna de IA versionada em /internal/v1.
- Streaming do Monitor Virtual via SSE.
- Banco relacional como fonte de verdade; vetor como índice derivado.
- Operações mínimas do serviço de IA: chat, indexar, remover e reindexar material.
- **Novo:** monitor humano fora do escopo atual; fica para uma fase futura.
- **Novo:** prática com questões e provas apenas em disciplinas que têm prova.
- **Novo:** seção de feedback da plataforma.

30. Decisões ainda em aberto

- Provedor de LLM e embeddings.
- Estratégia definitiva de armazenamento dos materiais (incluindo desenho do Google Drive, se adotado).
- Estratégia final de autenticação e transporte/armazenamento de tokens.
- Deploy e hospedagem de cada componente.
- Política quantitativa de rate limit.
- Parâmetros finais do RAG: modelo de embedding, tamanho/overlap dos chunks, top-k e critérios de suficiência.
- DER definitivo após retorno da equipe; mudanças podem impactar migrations e entidades JPA.

- **Novo:** formato das questões (múltipla escolha, discursivas ou ambas).
- **Novo:** se a primeira entrega gera questões novas por IA ou apenas exibe questões existentes.
- **Novo:** dificuldade das questões (os materiais não trazem essa classificação).
- **Novo:** tratamento de questões sem gabarito no material.
- **Novo:** identificação de disciplinas com prova (campo possui_prova ou dedução pelo material).
- **Novo:** formato do feedback da plataforma (nota, categorias, comentário e anonimato).
- **Novo:** processo de autorização dos professores para uso de listas e provas.
- **Novo:** períodos contemplados no MVP (a documentação trata do 1º ao 4º; o protótipo mostra mais).
- **Novo:** campos do perfil do usuário após a saída do monitor humano.

31. Próximos passos técnicos

1. Validar o DER final com a equipe, incluindo as propostas de prática e feedback.
2. Fechar o modelo lógico e migrations iniciais.
3. Criar os três projetos no repositório.
4. Configurar PostgreSQL + pgvector.
5. Implementar o esqueleto Spring Boot e Swagger.
6. Criar mocks para o React.
7. Criar o FastAPI mínimo e, depois, o pipeline RAG.
8. Estender a ingestão para listas de exercícios e provas e implementar a prática com questões.
9. Integrar streaming e fontes.
10. Executar testes de segurança, recuperação, fallback, prática e desempenho.

Conclusão

A arquitetura aprovada mantém o NEURON modular sem introduzir infraestrutura excessiva para o MVP. Spring Boot concentra o domínio e a segurança; React permanece desacoplado da IA; o serviço Python especializa o RAG e a prática com questões; e PostgreSQL + pgvector simplifica persistência e busca vetorial em uma única tecnologia de banco. A estrutura permite evolução posterior, inclusive a futura volta do monitor humano, sem tornar o Monitor Virtual responsável pelo restante do sistema.